

Laboratório 05 Sincronização e Memória Partilhada (parte 2)

Introdução

Uma das formas mais rápidas de comunicação entre processos é a utilização de memória partilhada. Quando dois processos estão a utilizar o mesmo segmento de memória partilhada, a comunicação ocorre diretamente escrevendo e lendo a memória, sem necessidade de chamar as rotinas do sistema operativo. No entanto, quando mais de um processo partilha o mesmo segmento de memória, é necessário garantir que quando se está a escrever na memória, nenhum outro processo tentará ler ou escrever para essa mesma memória. Por isso, é necessário sincronizar estes processos de acesso à memória. Um dos mecanismos que podem ser utilizados são os semáforos.

Revisão

Comece por rever, executar e compreender os exemplos fornecidos na aula 05.

Exercício 1. Conta Bancária Partilhada

Implemente um programa em C que defina uma macro de $N = 50$ processos, que irão aceder de forma concorrente a uma conta bancária, representada em memória partilhada. O saldo inicial da conta deverá ser de 1000 dinheiros e deverá igualmente ser usada uma macro `'#define SALDO 1000'`.

Cada processo deverá começar por imprimir o seu PID, de seguida esperar 50 microssegundos (função `usleep`) e por fim tentar fazer um levantamento de 2 dinheiros. Para isso, deverá verificar se o saldo disponível na conta, naquele instante, é suficiente. Caso seja, o valor é retirado do saldo. Caso contrário, o processo deverá indicar que não foi possível efetuar o levantamento por falta de saldo.

Depois de todos os processos tentarem levantar dinheiro e terminarem, o processo pai deverá imprimir o saldo final da conta, o saldo que era esperado e o número de levantamentos realizados com sucesso, libertando de seguida todos os recursos alocados.

Numa primeira fase, implemente a solução sem qualquer mecanismo de sincronização e observe os resultados. Numa segunda fase, adicione um mecanismo de exclusão mútua que garanta a correção das operações sobre a conta bancária (sugestão: crie uma estrutura composta pelo menos pelo saldo e por 1 semáforo unnamed. Coloque esta estrutura em memória partilhada).

Exercício 2. Casa da Sorte (c/ processos)

Reveja o enunciado do exercício 1 do Lab04 (Casa da Sorte) e analise o código fornecido (`lab04ex1bticketsfinal.c`). Pretende-se adaptar o código de maneira a utilizar processos (`fork()`) em vez de tarefas. Assegure-se que no final todos os recursos são libertados.

Exercício 3. Partilha de Mensagem

Desenvolva **dois** programas, um **escritor** e um **leitor**, que irão partilhar uma mensagem entre si.

O **programa escritor** deverá começar por criar uma zona de memória partilhada, com chave 123456 e tamanho 1024 bytes.

```
shm_id = shmget(123456, 1024, 0660|IPC_CREAT);  
shm_ptr = (char *) shmat(shm_id, NULL, 0);
```

De seguida, deverá recolher do utilizador uma frase curta utilizando o comando `fgets`:

```
printf("Insert message: ");  
fgets(shm_ptr, 20, stdin);
```

Por sua vez, o **programa leitor**, deverá aceder à frase na memória partilhada e simplesmente imprimir o seu conteúdo no ecrã.

Na altura da execução terá de abrir 2 janelas de terminal: numa lançará o programa escritor e na outra lançará o leitor.

Numa primeira abordagem, pode recorrer à instrução `sleep(20)` no programa escritor logo após a recolha da frase. Esta espera de 20 segundos dará tempo suficiente para lançar o programa leitor e este aceder à memória partilhada, para escrever a frase no ecrã (o leitor poderá depois colocar-se, ele próprio, também em `sleep`).

Após garantir que os programas têm um comportamento perto do esperado, retire os comandos `sleep` e recorra a semáforos para sincronizar os processos:

- O processo **escritor** deve bloquear após escrever a mensagem na memória partilhada e apenas poderá ser desbloqueado após o **leitor** escrever essa mensagem no ecrã.
- O processo **leitor** deve ficar bloqueado a aguardar que o escritor escreva uma mensagem na memória partilhada e só depois deve prosseguir.

O acesso à memória partilhada deve ser feito em exclusão mútua (e.g. semáforo binário, estilo mutex).

No final, ambos os programas devem assegurar que libertam todos os recursos.